ОТЧЁТ
ПО ДОМАШНЕЙ РАБОТЕ № 5

«Kubernetes ресурсы, планирование и параллельный MapReduce»

Индивидуальный вариант № 11 - DevOps-инженер

Выполнил: Алёшкин Дмитрий Игоревич
Группа: J3311
ИСУ: 464975

2026

1. Архитектура и среда исполнения

RustFS предоставляет через Service исходный CSV, фрагменты и результаты. Job split формирует фрагменты; Indexed Job запускает три mapper Pod; Job reduce объединяет частоты и сверяет их с ДЗ №3. StatefulSet RustFS использует четыре PVC.

Компонент	**Фактическая конфигурация**
Хост	Windows 11, Docker Desktop / WSL2, Linux Engine
kind / Kubernetes	0.33.0 / 1.35.8, один узел control-plane
ОС узла / runtime	Debian 13 / containerd 2.3.4
Ядро	6.6.87.2-microsoft-standard-WSL2
Capacity и allocatable Kubernetes	16 CPU; 14 211 260 KiB памяти; 110 Pod
Внешний предел Docker для узла	4 CPU; 5 GiB RAM; memory-swap = memory
Namespaces	devops-compute; devops-scheduling; devops-quota

kind выбран для изолированного воспроизводимого локального стенда без включения встроенного Kubernetes Docker Desktop. Одного узла достаточно для OOM, ограничения размещения, preemption, квот и нескольких параллельных Pod. Это не проверка распределённой отказоустойчивости между хостами.

Два разных уровня ограничений

Capacity/allocatable Kubernetes отражают ресурсы родительской VM. Внешняя cgroup Docker ограничивает реальное исполнение узла 4 CPU и 5 GiB. Поэтому резервирование в CPU-эксперименте рассчитывается по 16 CPU планировщика; его нельзя интерпретировать как наличие 16 CPU вычислительной мощности у контейнера узла.

Финальный аудит подтвердил Ready узла, работоспособность системных Pod и отсутствие MemoryPressure, DiskPressure и PIDPressure. Посторонние Kubernetes-ресурсы не использовались.

2. Контейнеризация и запуск Job

Образ содержит Python 3.11, S3-клиент, неизменённый mapper, адаптер и эталон пар. DuckDB и исходный CSV в вычислительный образ не включены. Базовый Python-образ зафиксирован по digest, все Python-зависимости закреплены точными версиями.

Параметр	**Реализация**
Команда	python kubernetes/worker.py single | split | map | reduce
Пользователь	UID/GID 10001, без root
Конфигурация	RUSTFS_ENDPOINT, S3_BUCKET, RUN_PREFIX, PARTS, JOB_COMPLETION_INDEX
Ключи	Secret rustfs-auth через envFrom; не включены в образ или журналы
Логи	JSON: начало, контрольные числа, время, результат сверки, memory.peak
Ресурсы обычной Job	requests: 250m CPU / 64 MiB; limits: 1 CPU / заданная память
Повторные запуски	restartPolicy: Never; backoffLimit: 0; deadline: 600 с
Выход при ошибке	Исключение и ненулевой exit code; расчёт не объявляется успешным

Практическая проверка образа

Локальный Docker-запуск обработал полный CSV за 29,33 с чистого вычислительного участка, с пиком 55,65 MiB. Все частоты совпали. Эти времена не сравниваются как benchmark с Kubernetes: различаются сеть, кэш и условия запуска.

После фиксации зависимостей финальный образ повторно собран и проверен Job verify-20261030-01 с лимитом 96 MiB: полный расчёт завершился, совпадение подтверждено, пик 56,17 MiB.

Конфигурация Dockerfile ограничивает build context через отдельный Dockerfile.dockerignore. Секреты не передаются аргументами сборки. Итоговый текстовый аудит проверяет отсутствие реальных ключей в новых файлах.

3. RustFS внутри Kubernetes

Использована версия RustFS 1.0.0 и схема предыдущей работы: четыре каталога /data/rustfs0…3, erasure set из четырёх дисков, EC:2. В Kubernetes каждому каталогу соответствует отдельный PVC объёмом 3 GiB. StatefulSet управляет Pod rustfs-0; ClusterIP Service предоставляет endpoint http://rustfs:9000.

Настройки передаются ConfigMap, учётные данные генерируются отдельно для кластера и передаются Secret. Для данных заданы fsGroup/runAsUser/runAsGroup 10001, для журналов выделен emptyDir. Startup/liveness проверяют /health, readiness - /health/ready. RustFS имеет requests 250m CPU / 256 MiB и limits 1500m CPU / 1 GiB.

Миграция без копирования внутренних дисков

Файл прочитан из исходного RustFS через S3 API и загружен в новое хранилище multipart-загрузкой. Исходные диски и объект не удалялись. Через временный localhost port-forward проверены размер и полный SHA-256; вычислительные Job используют Service внутри кластера.

Проверка	**Источник**	**Kubernetes**
Размер, байт	847619384	847619384
SHA-256	Совпадает полностью	Совпадает полностью
После замены Pod	Исходный объект сохранён	Хэш неизменен

Проверка сохранения данных

Pod rustfs-0 удалён штатной командой, StatefulSet создал новый Pod с другим UID и прежними PVC. После готовности хранилища полный SHA-256 CSV вычислен заново и совпал с исходным. Это подтверждает сохранение данных при замене Pod.

Ограничение: четыре PVC находятся на одном физическом хосте. EC:2 здесь не доказывает устойчивость к потере хоста. Данные local-path сохраняются при замене Pod, но не гарантированы после удаления kind-кластера. Внешний оригинал сохранён.

4. OOMKilled и подбор памяти

Сначала полный расчёт запущен с достаточным лимитом 256 MiB. Пик измерен чтением cgroup v2 memory.peak после вычислений и записи результата. Затем лимит уменьшен до 28 MiB, и Kubernetes зафиксировал OOMKilled. Перезапуски отключены, поэтому исходный отказ не скрыт повторной попыткой.

Запуск	**Req. RAM**	**Limit RAM**	**Пик, MiB**	**Исход**
Измерение	64 MiB	256 MiB	56,14	Succeeded
Нехватка	16 MiB	28 MiB	Недоступен*	OOMKilled / 137
Подбор	64 MiB	96 MiB	56,03	Succeeded
Финальный образ	64 MiB	96 MiB	56,17	Succeeded

*Процесс был завершён при превышении лимита памяти и не успел вывести итоговый memory.peak. Поэтому измеренное значение пика для этого запуска отсутствует

Обоснование рабочего предела

Рабочий предел = округление вверх до 16 MiB значения (56,1445 × 1,5) = **96 MiB**. Это запас относительно измеренного пика, а не гарантия для произвольного другого датасета. Повторный расчёт с 96 MiB успешно завершился, подтвердил все частоты и отсутствие OOMKilled.

Почему нельзя приравнять к 87,97 MiB из ДЗ 3

Windows Peak Working Set измеряет максимальный резидентный набор страниц процесса. memory.peak отражает максимум учтённой памяти cgroup и её потомков; учёт включает не только пользовательские страницы, но и относящиеся к группе кэш/память ядра. Это разные метрики.

Кроме того, старый расчёт удерживал глобальные счётчики размеров 1, 2 и 3, а адаптер удерживает глобально только пары. Поэтому более низкое значение не доказывает превосходство Kubernetes или оптимизацию исходного алгоритма.

Пик считывается перед выходом: это максимум до момента чтения. Для Job split отдельно установлен 1 GiB: она записывает временные файлы и имеет другой профиль памяти, включая файловый кэш. Её пик составил 928,27 MiB.

5. Параллельные вычисления и сверка

Job split прочитала CSV парсером и распределила целые записи по индексам n mod 3. В каждый фрагмент сохранены все исходные поля. Переносы строк внутри кавычек не считаются отдельными вакансиями. Для каждой части сохранены число записей и SHA-256 их канонического представления.

Индекс	Все записи	DevOps	С навыками	Вхождения пар
0	76463	1693	1179	46560
1	76463	1732	1169	42890
2	76462	1733	1186	45336

Indexed Job задана с completions=3 и parallelism=3. Каждый Pod получает JOB_COMPLETION_INDEX и читает только свой объект. Перед расчётом используется барьер готовности. Все три интервала именно обработки данных пересекались на **4,915 с**; длительности отдельных mapper составили 5,09-5,28 с.

Reducer прочитал по одному частичному результату каждого индекса, сложил частоты и сравнил весь словарь с эталоном. Итог: 229 388 записей, 5 158 DevOps, 3 534 с навыками, 22 311 пар и 134 786 вхождений. Missing=0, extra=0, different=0.

Топ-10 эталон ДЗ 3 Kubernetes

Навык 1	Навык 2	ДЗ №3	Kubernetes
Docker	Kubernetes	1221	1221
Docker	Linux	1190	1190
Kubernetes	Linux	1143	1143
Ansible	Linux	863	863
DevOps	Linux	825	825
DevOps	Kubernetes	800	800
DevOps	Docker	798	798
Ansible	Docker	774	774
Ansible	Kubernetes	756	756
Linux	Python	738	738

Сортировка топ-10: по частоте убывающе, затем по паре лексикографически. Общего ускорения конвейера не заявляется: предварительное разбиение, S3 I/O и запуск Pod также требуют времени.

6. CPU scheduling и scheduler preemption

Ожидание по requests, а не загрузке процессора

Узел предоставляет scheduler 16000m CPU. Уже зарезервировано 1200m. Блокирующий Pod запрашивает 14300m, оставляя 500m. Следующая вычислительная задача запрашивает 1000m: scheduler не может разместить её и выдаёт Insufficient cpu.

Блокирующий процесс выполняет sleep, поэтому большая величина requests не создаёт искусственной нагрузки на ноутбук. После удаления только этого Pod ожидавшая задача запускается, вычисляет сумму квадратов и завершается. UID до и после одинаковый: это тот же Pod, а не новая попытка.

Фаза	**Наблюдение**	**Вывод**
Резервирование	Блокирующий Pod Running	CPU requests заняты
Дополнительная задача	Pending; Insufficient cpu	Нет места по requests
Ресурсы освобождены	Тот же UID; Succeeded	Планировщик смог разместить задачу

Вытеснение приоритетом

Созданы PriorityClass devops-low=-100 и devops-high=-50. Оба значения ниже обычного приоритета RustFS (0), чтобы эксперимент не вытеснял хранилище. Низкоприоритетный Pod сначала работает и резервирует свободные CPU; высокоприоритетному требуется 1000m, которых без вытеснения не хватает.

Scheduler выбрал низкоприоритетный Pod и записал событие **Preempted**, reportingComponent=**default-scheduler**. Затем высокоприоритетная вычислительная задача завершилась.

Фрагмент сообщения события:
Preempted by pod ae551717-1b53-4a27-bbf3-301240db7831
on node devops-k8s-control-plane

Это именно scheduler preemption. OOMKilled определяется ограничением памяти, a node-pressure eviction - давлением на узел; ни одно из этих событий не заменяет доказательство Preempted от scheduler.

7. ResourceQuota и различия механизмов

В отдельном namespace devops-quota создана ResourceQuota budget. Первый Pod занимает часть квоты. Попытка создать второй Pod превышает суммарные requests CPU/памяти и limits памяти; API отклоняет запрос до появления Pod.

Ресурс	Hard	Первый Pod	Второй Pod	Сумма
requests.cpu	500m	250m	400m	650m > 500m
requests.memory	128 MiB	64 MiB	96 MiB	160 MiB > 128 MiB
limits.cpu	1 CPU	250m	400m	650m <= 1 CPU
limits.memory	256 MiB	128 MiB	192 MiB	320 MiB > 256 MiB
pods	3	1	1	2 <= 3

Наблюдаемый отказ и восстановление

API вернул **Forbidden** с сообщением **exceeded quota: budget**. Проверка get pod подтвердила, что отклонённый Pod не существует. Это не Pending: объект не прошёл admission.

После удаления первого экспериментального Pod скрипт дождался освобождения quota used и отправил неизменённую спецификацию второго. Она была принята, Pod перешёл в Running. Затем тестовый Pod удалён; ResourceQuota и доказательства сохранены.

Ситуация	Где действует	Что наблюдалось
Лимит памяти	cgroup / kernel	Процесс убит; OOMKilled
Недостаток CPU requests	Scheduler	Созданный Pod остаётся Pending
Приоритет	Scheduler	Низкий приоритет вытеснен
Превышение ResourceQuota	Admission API	Создание Pod отклонено

ResourceQuota учитывает суммы объявленных requests/limits. Она не измеряет текущую загрузку CPU и не гарантирует физическое наличие ресурсов на узле. Поэтому прохождение квоты само по себе не означает возможность scheduling.

8. Kubernetes, YARN и Mesos

Сравнение выполнено по официальной документации. Это теоретическая часть; YARN и Mesos в данной работе не разворачивались. Их механизмы не являются прямыми взаимозаменяемыми аналогами.

Аспект	**Kubernetes**	**YARN**	**Mesos**
Архитектура	API, scheduler, controllers, kubelet	ResourceManager, NodeManager, ApplicationMaster	Master, agents, frameworks
Распределение	Scheduler выбирает узел для Pod	RM выдаёт ресурсные containers по заявкам AM	Master предлагает ресурсы; framework выбирает задачи
CPU / память	requests для размещения; limits для исполнения	Ресурсный container: memory/vcores; enforcement зависит от NM	Task resources, allocator, isolators; исполнение зависит от конфигурации
Приоритеты	PriorityClass для Pod	Приоритеты приложений и политика очередей	Политика allocator и framework; не единый PriorityClass
Вытеснение	Удаление менее приоритетных Pod ради размещения	CapacityScheduler может возвращать ресурсы очередям	QoS может убивать/throttle revocable tasks; не тот же механизм
Квоты	ResourceQuota: суммы в namespace при admission	Ёмкости и пределы очередей, пользовательские лимиты	Квоты scalar-ресурсов на роль
Batch	Job / Indexed Job	MapReduce и другие приложения через AM	Планирование batch через framework
Отказы	Контроллеры восстанавливают workload; HA control plane отдельно	AM следит за задачами; RM HA отдельно	HA master + восстановление framework / task reconciliation
Применение	Сервисы и контейнерные batch-задачи	Экосистема Hadoop, аналитические очереди	Исторически совместное использование кластера frameworks

YARN container - выделение ресурсов, не обязательно OCI/Docker-контейнер. Mesos использует двухуровневое распределение ресурсов; восстановление задач требует логики framework. В актуальной официальной документации Mesos обозначен как retired, поэтому его сравнение здесь имеет архитектурный и учебный смысл.